

CURLIX

System Design Document v1.0 — Anonymous Scalable Architecture

This document defines the internal architecture, system behavior, scaling strategy, caching model, API structure, and deployment workflow for Curlix — a scalable URL shortening platform built entirely using free infrastructure.

1. System Overview

Curlix is designed as a distributed backend-focused application optimized for: Fast redirects Low database load Asynchronous analytics processing Anonymous ownership without authentication Free-tier scalability Core architectural pattern: Redis-first lookup + queue-based analytics + PostgreSQL persistence.

2. Frontend Architecture

Framework: React + Vite

Frontend Responsibilities: Short URL creation interface Analytics dashboard rendering QR code display and download Owner token storage in localStorage API communication with backend **Routing:** / → Home page /dashboard → User link list /analytics/:code → Analytics view **UI Stack:** Tailwind CSS React Router Recharts for analytics visualization

3. Backend Architecture

Framework: Node.js + Express

Core Modules: Controllers → Request handling Services → Business logic Queue Workers → Analytics processing Middleware → Validation, rate limiting, CAPTCHA verification Redis Layer → Cache and queue backend **Main Redirect Flow:** User requests short URL Redis cache checked first If cache miss → PostgreSQL lookup Analytics job added to BullMQ queue 302 redirect returned immediately Redirect latency target: **<10ms on cache hit**

4. Database Schema

Table	Purpose
urls	Stores URL mappings and ownership data
clicks	Stores analytics click events

URLs Table: id short_code long_url owner_token_hash created_at expires_at is_active **Clicks Table:** id short_code timestamp ip_hash device referrer Indexes: urls(short_code) clicks(short_code, timestamp)

5. Anonymous Ownership Flow

Curlix intentionally avoids traditional signup/login systems. Instead of JWT authentication: Each created link generates a unique owner token The token is shown once to the creator Backend stores only the hashed token Edit/delete operations require the owner token Frontend stores token locally in browser localStorage Benefits: No signup friction No session management Lower infrastructure complexity Reduced attack surface

6. API Structure

Method	Endpoint	Purpose
POST	/api/shorten	Create short URL
GET	/:code	Redirect user
GET	/api/analytics/:code	Fetch analytics
DELETE	/api/links/:code	Delete link

7. Caching Strategy

Technology: Upstash Redis

Caching strategy: All redirects checked against Redis first Cache stores short_code → long_url mappings TTL set to 24 hours Cache invalidated manually on delete/update Target cache hit rate: >85% Benefits: Reduces PostgreSQL reads dramatically Improves redirect performance Lowers infrastructure cost

8. Queue-Based Analytics

Technology: BullMQ + Redis

Analytics processing is fully asynchronous. Workflow: User clicks short link Redirect returned immediately Analytics job pushed to queue Worker processes click data separately Benefits: No redirect slowdown Independent worker scaling Improved user experience

9. Security Architecture

Security layers implemented: Cloudflare Turnstile CAPTCHA Redis-backed rate limiting Helmet.js security headers URL validation and sanitization Reserved keyword blocking SHA-256 IP hashing HTTPS enforcement Environment variable protection Anti-abuse measures: Per-IP create limits Honeypot bot fields Device fingerprint checks

10. Deployment Strategy

Component	Deployment Platform
Frontend	Vercel

Backend API	Render
Database	Supabase
Redis	Upstash
Monitoring	UptimeRobot

Deployment Notes: Render cold starts prevented using UptimeRobot Frontend and backend deployed independently
Environment variables configured securely per platform All infrastructure uses permanently free plans

Document prepared for Curlix Engineering — System Design v1.0